Classroom Attendance Prediction Using Academic Schedule and Historical Attendance Data

1. Project Overview and Executive Summary

Educational institutions often face irregular student attendance due to compounding factors such as lecture timing, day of the week, examinations, weather, holidays, and faculty schedules. While most institutions systematically record attendance, they rarely utilize this data to predict future attendance patterns proactively.

The objective of this capstone project is to develop a predictive machine learning pipeline capable of estimating the attendance percentage or the expected number of students in a classroom using historical attendance records and academic scheduling information. By executing this project, students will create an original dataset and map theoretical data science concepts to practical academic planning.

The primary objectives are to:

- Create a custom, physically verified attendance dataset.
- Identify and quantify the factors affecting student attendance.
- Predict attendance for future scheduled lectures.
- Help faculty proactively identify classes likely to have low attendance.
- Support academic planning and optimize classroom resource allocation.

2. Phase 1: Rigorous Data Collection Methodology

The foundation of this predictive model relies entirely on original, organically collected data. Students must manually collect attendance data from one or more classes over a full semester to construct a robust dataset. **No public datasets may be used for this capstone. Use historical data.**

2.1 Collection Parameters and Integrity

To capture a statistically significant representation of academic attendance, student must conduct data collection over a recommended period of **1 Week**.

- **Scope:** Data should span multiple subjects and multiple divisions/sections, capturing 3 to 10 lectures per day.
- **Volume:** This systematic collection typically results in 500 to 3,000 records, depending on the scale of the tracked classes.
- **Sources:** Data must be aggregated after every lecture using faculty attendance registers, manual head counts, department attendance sheets, Learning Management Systems (LMS), timetable records, and the academic calendar.

Data Privacy Challenge: Student must secure explicit permission to access departmental attendance records. All data must be strictly anonymized; avoid storing student names or roll numbers unless absolutely necessary and authorized by the administration.

2.2 The Data Dictionary

During observation periods, precision and consistency are critical. Each record must include, but is not limited to, the following features:

Feature	Description	Example / Domain
Date	Exact date of the lecture	DD-MM-YYYY
Day of Week	Operational day	Monday–Saturday
Lecture Number	Sequential slot in the timetable	1st, 2nd, 3rd, etc.
Start Time	Timestamp of class commencement	9:00 AM
Subject	Specific subject name or code	CS101, Python, DBMS
Faculty ID	Encoded identifier for the instructor	F_012
Semester	Current academic semester	1–8
Branch	Academic department	CSE, IT, Mechanical, etc.
Section	Sub-division of the cohort	A, B, C
Classroom	Physical room identifier	Room 402
Total Enrolled Students	Maximum capacity/class strength	Integer (e.g., 60)
Students Present	Actual observed attendance count	Integer (e.g., 45)

Feature	Description	Example / Domain
Attendance Percentage	(Present ÷ Enrolled) × 100	Continuous / Float
Previous Lecture Attendance	Attendance in the preceding class	Continuous / Float
Gap Since Previous Lecture	Time elapsed between classes	Hours or Days
Practical/Theory	Pedagogical format of the session	Categorical
Internal Test Week	Proximity to examinations	Yes / No
Assignment Due	Deadline coincidence	Yes / No
Holiday Before/After	Proximity to academic breaks	Yes / No
Weather (Optional)	Environmental conditions	Sunny, Rainy, Cloudy
Special Event (Optional)	Campus festivals or seminars	Yes / No
Faculty Experience (Optional)	Instructor tenure	Years

3. Phase 2: Data Engineering and Exploratory Analysis

Once raw data is logged, it must be digitized and transformed into a machine-readable CSV format. This phase focuses on structuring, cleaning, and engineering new predictive signals.

3.1 Data Cleaning and Handling Missing Values

Real-world academic data contains anomalies. Student must account for missing attendance entries resulting from sudden faculty absences, ad-hoc timetable changes, or differences in attendance recording methods across various departments.

3.2 Feature Engineering

To maximize model accuracy, students must derive advanced temporal and historical features from the base dataset. Recommended engineered features include:

- Day of the semester and Week number.
- Days elapsed since the last holiday.
- Consecutive lecture count for the student cohort on that day.
- Rolling average attendance of the previous 3 lectures.
- Monthly average attendance and macro-attendance trends.

- Time-of-day clustering (e.g., Morning vs. Afternoon, Before Lunch vs. After Lunch).
- Binary flag for the "Week before examination."

3.3 Example Dataset Structure

Below is a sample representation of how the engineered dataset should look prior to modeling:

Date	Time	Subject	Semester	Day	Previous %	Test Week	Holiday Tomorrow	Attendance %
05-08	09:00	Python	3	Monday	78	No	No	**82**
05-08	11:00	DBMS	5	Monday	69	Yes	No	**91**
06-08	08:00	AI	7	Tuesday	84	No	Yes	**63**

4. Phase 3: Algorithmic Modeling and Implementation

The modeling phase requires student to define their target variables carefully, as this problem can be framed as either a regression or classification task.

4.1 Target Variable Definition

- **Regression Approach:** Predict the continuous exact *Number of students present* or the *Attendance percentage*.
- **Classification Approach:** Predict categorical attendance bands, such as:
  - Low (<50%), Medium (50–75%), High (>75%)
  - Poor, Average, Good

4.2 Advanced Algorithmic Training

Student will leverage Scikit-learn (and optionally XGBoost/CatBoost) to train and compare multiple algorithms.

Regression Algorithms:

- Linear Regression
- Decision Tree Regressor
- Random Forest Regressor
- Gradient Boosting
- XGBoost / CatBoost

Classification Algorithms:

- Logistic Regression
- Decision Tree & Random Forest
- Support Vector Machine (SVM)
- k-Nearest Neighbors (k-NN)
- XGBoost
- Naïve Bayes

5. Phase 4: Experimental Rigor and Evaluation

Model evaluation ensures the algorithm generalizes to future semesters and timetable shifts.

5.1 Quantitative Evaluation Metrics

Depending on the chosen target variable formulation, models must be evaluated using standard metrics.

- **For Regression Models:** Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Mean Absolute Percentage Error (MAPE), and R^2 Score.
- **For Classification Models:** Accuracy, Precision, Recall, F1 Score, and ROC-AUC.

6. Phase 5: Software Engineering and Deployment

The final phase introduces MLOps principles, requiring the transition of the Jupyter Notebook pipeline into a functional deployment using Python, Pandas, and NumPy.

6.1 Web Application and Visualization

Student must deploy an interactive dashboard using **Streamlit** or **Power BI**. The deployed application should serve as an analytical tool for department heads and faculty, enabling them to:

- Predict attendance for upcoming scheduled lectures.
- Identify time slots with consistently low attendance.
- Highlight specific subjects that tend to suffer from poor attendance.
- Estimate the negative/positive impact of upcoming tests, holidays, or timetable shifts.

6.2 Possible Extensions for High Achievers

To elevate the capstone, student can implement the following extensions:

- Compare attendance prediction efficacy across entirely different engineering departments.
- Develop individualized absenteeism predictions for specific core courses.
- Build an automated recommendation engine that suggests optimal lecture timings based on historical peak attendance hours.
- Analyze and visualize the direct statistical influence of weather and examinations on cohort behavior.

7. Final Deliverables and Submission Protocol

Upon completion of the project lifecycle, student must submit a comprehensive portfolio. Ensure your submission archive contains:

- **Handwritten data-collection logbook:** Physical proof of the primary data collection effort.
- **Raw and cleaned CSV datasets:** Both iterations of the data to reproduce the engineering pipeline.
- **Source code:** Modularized Jupyter Notebooks detailing EDA, feature engineering, and model training.
- **Experiment table:** A matrix detailing the configurations and outcomes of all tested algorithms.
- **Deployment files:** All requisite code for the Streamlit dashboard or Power BI file.
- **Working application demonstration:** A functional presentation of the user interface executing predictions.
- **Final report:** A cohesive document synthesizing the analytical findings, optimal scheduling recommendations, and deployment strategy.